



# EMERGE-Policy: A Robot Mind Emerges Beyond a Single Policy

Zhirui Fang<sup>\*,1</sup>, Qingchi Yu<sup>\*,2,†</sup>, Ziyang Chen<sup>\*,1</sup>, Longfei Li<sup>\*,†</sup>, Haoran Ma<sup>3,†</sup>, Keru Zhou<sup>1</sup>, Xinrun Xu<sup>1</sup>, Samith Va<sup>1</sup>, Yuxuan Hu<sup>4</sup>, Qiang Du<sup>1</sup>, Peixuan Song<sup>1</sup>, Hailong Shang<sup>1</sup>, Yongkang Deng<sup>5,†</sup>, Xin Li<sup>5,†</sup>, Yezhen Wang<sup>1</sup>, Zhe Li<sup>1</sup>, Hao Luo<sup>6</sup>, Shuyan Li<sup>1</sup>, Ziwei Wang<sup>7</sup>, Weijian Deng<sup>1</sup>, Xiu Li<sup>1,✉</sup>

<sup>1</sup>Tsinghua University   <sup>2</sup>Nanjing University of Science and Technology   <sup>3</sup>Xi'an Jiaotong University  
<sup>4</sup>Xidian University   <sup>5</sup>Harbin Institute of Technology   <sup>6</sup>Peking University   <sup>7</sup>Nanyang Technological University

\*Equal contribution   †Work done during the internship at Tsinghua University   ✉Corresponding author



[Website](#)

[Code](#)

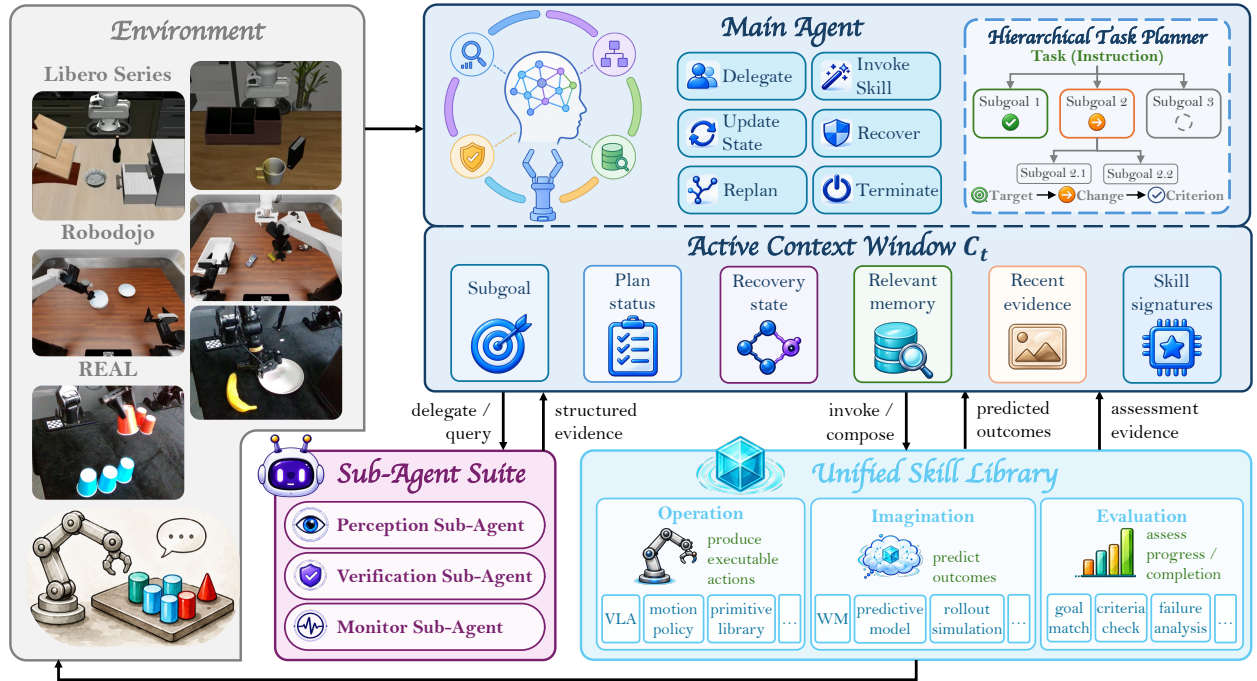


Figure 1: **System overview of EMERGE-Policy and its coordinated components.** The Main Agent coordinates Sub Agents with Operational, Imagination, and Evaluation Skills. Structured evidence guides capability selection, verification, and localized recovery across simulation and embodied execution, producing a system-level policy beyond any single backend.

## Abstract

A robot’s effective “mind” need not reside in a single policy. It can emerge when specialized components perceive, reason, predict, act, verify, and remember within a shared orchestration process. EMERGE-Policy turns this perspective into a graph-structured agentic framework that coordinates both capability invocation and information exchange. A Main Agent retains task-level state within an active context window, while role-specific Sub Agents process perception, execution monitoring, verification, and memory consolidation in isolated contexts and return structured, task-relevant evidence. Role-specific contexts control information load by exposing only decision-relevant evidence to the Main Agent, while the functional Skill interface composes heterogeneous backends as Operational, Imagination, and Evaluation Skills. Criterion-grounded verification, textual failure diagnosis, and Branch Stack recovery provide localized correction, with token-aware external memory preserving task-relevant state. Together, their closed-loop interaction realizes the system-level policy captured by the name EMERGE-Policy. Without additional fine-tuning, we achieved outstanding performance on several public benchmark that have had a wide-reaching impact, and conducted a series of real robot experiments. These system-level results suggest that through the division of different functional sub-tasks among multiple agents and their concurrent collaboration, as well as the technical paradigm where the model is regarded as a skill and called within the framework, EMERGE-Policy can extend the robust robot policies beyond isolated runs.

## 1 Introduction

*“Thinking outside the brain means skillfully engaging entities external to our heads.”*

—Annie Murphy Paul, *The Extended Mind*<sup>[1]</sup>

For embodied agents, this observation has a concrete systems interpretation: intelligence is not confined to a single policy or reasoning model, but is realized through the coordinated interaction of perception, prediction, task reasoning, physical control, external memory, and feedback from the environment. This coordination becomes particularly consequential in robot manipulation, where a robot must repeatedly ground an instruction in the current scene, select and execute an appropriate behavior, verify the resulting physical state, and respond when execution deviates from the plan. Because an error at one stage can invalidate the assumptions of subsequent stages, reliable behavior depends not only on the capability of individual components, but also on how information and control are organized across the complete system. The central challenge is therefore to coordinate specialized capabilities while controlling how much information the high-level planner must process at each decision step.

Existing work provides several complementary components for this process. Vision-Language-Action models have progressed from transferring pretrained vision-language representations to robot control<sup>[2,3]</sup>, through diffusion- and flow-based action generation<sup>[4–8]</sup>, to dual-system designs that combine deliberative reasoning with reactive control<sup>[9]</sup>. Video-based pretraining introduces broader priors over human activities and physical dynamics<sup>[10–12]</sup>, while World-Action Models couple action generation with future-state prediction<sup>[13–15]</sup>. In parallel, Code as Policies<sup>[16]</sup> and RoboCodeX<sup>[17]</sup> represent robot behavior as executable programs over perception and control APIs; CaP-X<sup>[18]</sup> studies how abstraction, interaction, and grounding affect coding-agent reliability, and ASPIRE<sup>[19]</sup> converts validated repairs into reusable skills. More recently, RoboClaw<sup>[20]</sup>, HarnessVLA<sup>[21]</sup>, and RoboHarness<sup>[22]</sup> demonstrate how high-level agents can orchestrate learned policies,

analytical controllers, perceptual tools, verification, and execution memory. Although these systems differ in scope, their orchestration is commonly centered on capability selection and control transfer. Detailed outputs from perception, execution monitoring, verification, and memory processing can therefore be passed directly to the high-level planner, even when only part of that information is relevant to its next decision. This motivates processing such evidence separately and returning structured, decision-relevant results to the planner.

We introduce **EMERGE-Policy**, a graph-structured agentic policy framework for robot manipulation. Its name reflects how coherent agentic behavior can emerge from the interaction of specialized components rather than being assigned to a single model. This emergence is structured rather than incidental: EMERGE-Policy makes functional roles, information boundaries, capability invocation, verification, and recovery explicit, so that the coordinated system acts as a policy beyond any one component. A Main Agent maintains compact global task state, decomposes instructions into criterion-grounded subgoals, and coordinates the execution process. Specialized Sub Agents handle perception, execution monitoring, verification, and memory consolidation in isolated contexts, returning structured evidence rather than complete interaction histories. This organization limits interference from dense low-level evidence while retaining the information required for task-level decisions. By making evidence boundaries explicit, it also clarifies which observation or result informs each orchestration decision.

EMERGE-Policy connects this information-flow organization to action through a shared subgoal specification. Each subgoal defines a target entity, an intended state change, and an explicit completion criterion, providing a common contract for three complementary Skill roles. Operational Skills execute actions toward the intended change, Imagination Skills predict candidate outcomes, and Evaluation Skills assess feasibility, progress, and criterion satisfaction. Because these roles are defined by function rather than implementation, different policy and world-model backends can be composed within the same closed loop. After execution, a satisfied criterion advances the plan; otherwise, the system generates a textual diagnosis and pushes a localized recovery objective onto the Branch Stack. Once recovery succeeds, execution returns to the interrupted plan, while global replanning is reserved for failures that local correction cannot resolve. Token-aware external memory preserves task state and relevant evidence throughout this process. Together, these mechanisms control information load, compose heterogeneous capabilities, and localize correction; their closed-loop interaction produces the system-level policy captured by EMERGE-Policy.

We evaluate EMERGE-Policy on standard and perturbed LIBERO benchmarks and study its deployment in a multistep real-robot manipulation task. The experiments examine the complete orchestration system with different VLA and WAM execution backends, particularly in settings involving semantic grounding, outcome verification, and recovery. Our contributions are threefold:

- We introduce EMERGE-Policy, a graph-structured agentic framework in which robot behavior emerges from the coordination of a Main Agent, role-specific Sub Agents, Skills, and external memory rather than from a single policy. By adopting this approach, we divided the task into several parts. Different agents can independently complete the sub-tasks or sub-functions, thereby solving the overall problem.
- We have expanded the definition of skills. EMERGE-Policy regards the currently widely-discussed models such as VLA, World model, and Verifier model as the skills of the agent. These skills are invoked by the agent at different stages of completing the task. Furthermore, we define explicit information and control interfaces: Sub Agents return only structured, task-relevant evidence, while functional Skills, completion criteria, Branch Stack recovery, and episodic memory connect planning, action, and feedback.
- We conducted thorough and extensive experiments on a series of widely adopted benchmarks. The experiment shows that our framework has achieved a significant improvement compared to the most relevant models ( $\pi_{0.5}$  and Cosmos Policy). It has increased by 2.0% and 0.7% on LIBERO, by 2.6% and 11.7% on LIBERO-plus, and significantly enhanced the performance of memory on Robodojo, and increased the success rate by 3.59% compared to the baseline, and we deploy it on a physical robot for a series of long-horizon tasks.

## 2 EMERGE-Policy: Graph-Structured Agentic Policy Orchestration

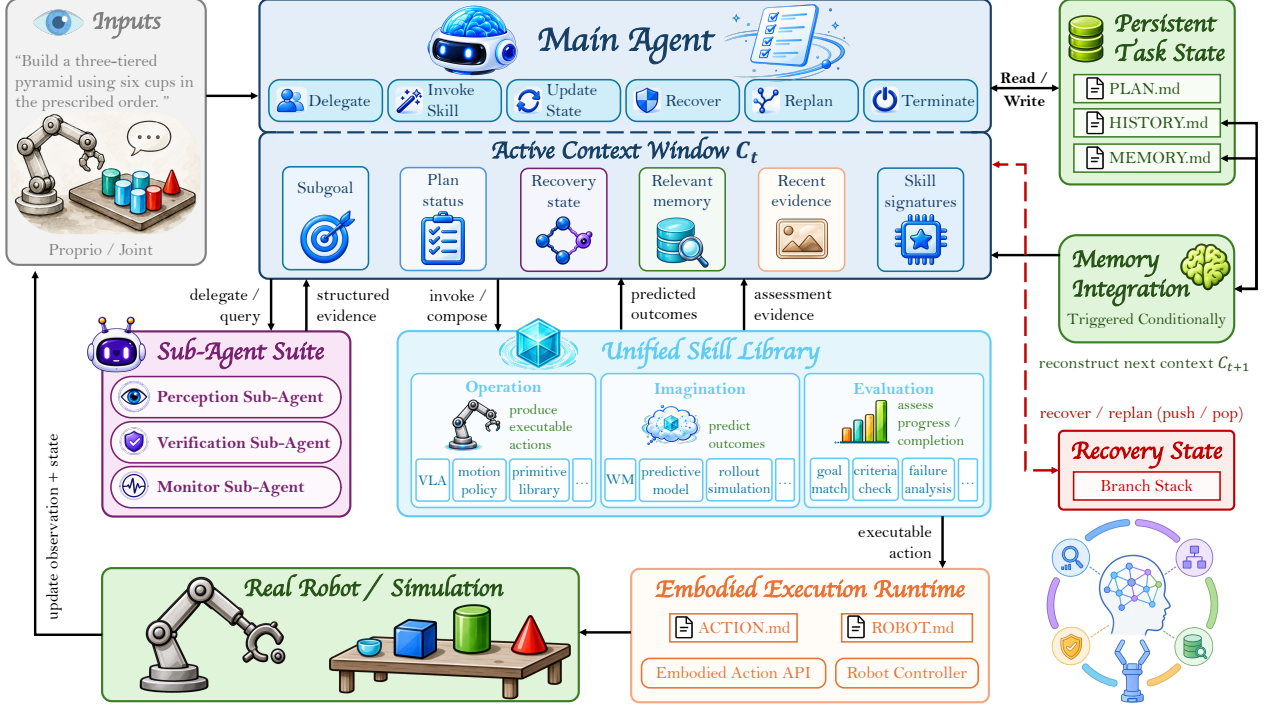


Figure 2: **EMERGE-Policy workflow from instruction to verified execution.** The Main Agent maintains active task context and coordinates role-specific Sub Agents with Operational, Imagination, and Evaluation Skills. Structured evidence and Skill outcomes update task state after each interaction: verified successes advance the plan, while failures enter localized Branch Stack recovery before global replanning. Episode memory preserves relevant task state throughout execution.

**Overview.** As shown in Figure 2, given a language instruction  $\mathcal{T}$  and the current observation  $\mathcal{O}_t$ , EMERGE-Policy organizes robot manipulation as closed-loop coordination among a Main Agent, role-specific Sub Agents, and a unified Skill library. The Main Agent maintains an active task context  $\mathcal{C}_t$ , decomposes the instruction into subgoals with explicit completion criteria, and determines which Sub Agent or Skill to invoke next.

Sub Agents process task-specific evidence for perception, execution monitoring, verification, and memory consolidation in isolated contexts, returning concise structured results to the Main Agent. The Skill library organizes heterogeneous capabilities by their roles in the decision loop: Operational Skills execute robot actions, Imagination Skills predict candidate outcomes, and Evaluation Skills assess feasibility, progress, or completion. Because Skills share this functional interface, heterogeneous backends can participate in the same orchestration protocol without assuming identical internal architectures. Following each physical execution, the observed outcome is evaluated against the active subgoal criterion. Successful verification advances the plan, whereas failure introduces a localized recovery subgoal into the Branch Stack. If local recovery remains unsuccessful, the Main Agent revises the high-level plan. External memory preserves task state and relevant evidence throughout this closed-loop process. Within this graph, role-specific contexts regulate information flow, the Skill interface composes heterogeneous backends, and criterion-based verification with the Branch Stack supplies localized correction. The system-level policy emerges from their closed-loop interaction, which determines how each module’s action or evidence shapes the next decision.

### 2.1 Main Agent: Planning, Recovery, and Memory

The Main Agent coordinates task-level planning, capability invocation, outcome verification, and failure



recovery. Given the instruction  $\mathcal{T}$  and evidence derived from the current observation  $\mathcal{O}_t$ , it maintains an active task context  $\mathcal{C}_t$ . The Main Agent implements an LLM-based orchestration policy  $\pi_M$  over a decision space  $\mathcal{D}$ . At interaction step  $t$ , the orchestration decision is given by

$$d_t \sim \pi_M(\cdot \mid \mathcal{C}_t), \quad d_t \in \mathcal{D}. \quad (1)$$

Here,  $\mathcal{D}$  comprises decisions to delegate evidence processing to a Sub Agent, invoke a Skill, update the task state, initiate recovery, revise the plan, or terminate execution. See the [Figure 3](#) for details.

### 2.1.1 Hierarchical Context Engineering

The active context  $\mathcal{C}_t$  contains the task instruction, current subgoal, plan status, top Branch Stack entry, task-relevant memory, recent structured results, and compact signatures of the available Skills. Raw multimodal observations and complete execution traces are kept outside this context unless required for the current decision. EMERGE-Policy controls context growth in three ways. First, stable constraints and embodiment configurations are separated from dynamic task state and inserted only when relevant. Second, the Main Agent initially receives only compact Skill signatures; complete parameter schemas are retrieved when a Skill is selected for invocation. Third, evidence-intensive perception, monitoring, verification, and memory operations are delegated to Sub Agents operating in isolated contexts. Each Sub Agent receives a task-specific request and returns a structured result rather than its complete processing history.

### 2.1.2 Hierarchical Task Planner and Branch Recovery

The Main Agent decomposes the instruction  $\mathcal{T}$  into an ordered sequence of executable subgoals. Let  $g_i$  denote the  $i$ -th subgoal in this sequence:

$$g_i = \langle \text{Target}_i, \text{Change}_i, \text{Criterion}_i \rangle. \quad (2)$$

where  $\text{Target}_i$  identifies the relevant entity,  $\text{Change}_i$  describes the intended state transition, and  $\text{Criterion}_i$  specifies the condition for verifying completion. The resulting subgoal sequence and its execution status are maintained in `PLAN.md`.

After a subgoal is executed, its execution outcome is evaluated against its completion criterion. If the criterion is satisfied, the Main Agent marks the subgoal as complete and proceeds to the next one. Otherwise, the subgoal is marked as incomplete and execution advances. When verification detects a failure, the Main Agent generates a textual diagnosis of the observed discrepancy, inspired by the verbal-reflection mechanism of Reflexion<sup>[23]</sup>. Based on this diagnosis, the Main Agent formulates a localized recovery subgoal and pushes it onto the Branch Stack. The stack-top recovery subgoal is addressed first; upon success, it is removed from the stack. Once it is completed, the branch is removed and execution resumes from the interrupted plan. If a recovery subgoal continues to fail after a preset number of attempts (*e.g.*, two), the Main Agent formulates an alternative a few times (*e.g.*, twice) in a row, then recovery subgoal and pushes it onto the Branch Stack. The Main Agent revises the high-level plan.

### 2.1.3 Token-Aware Memory Consolidation Engine

EMERGE-Policy maintains four complementary forms of task state, together with a separate recovery stack. The active context window  $\mathcal{C}_t$  serves as the Main Agent’s working memory. `PLAN.md` stores the current subgoal sequence and completion status. `HISTORY.md` is an append-only chronological archive of summarized observations, actions, and outcomes. `MEMORY.md` is a consolidated belief state containing task-relevant spatial, environmental, and procedural facts; unlike the chronological history, it may be revised when later evidence changes an earlier belief. The Branch Stack separately stores transient recovery subgoals.

Memory consolidation is triggered when the active context window reaches a token capacity. A Memory Integration Agent then appends the evicted chronological summary to `HISTORY.md`, merges relevant facts into `MEMORY.md` while revising stale entries, and reconstructs the next active context  $\mathcal{C}_{t+1}$  from the instruction, active `PLAN.md` state, Branch Stack, relevant consolidated facts, and recent Sub-Agent results. The full

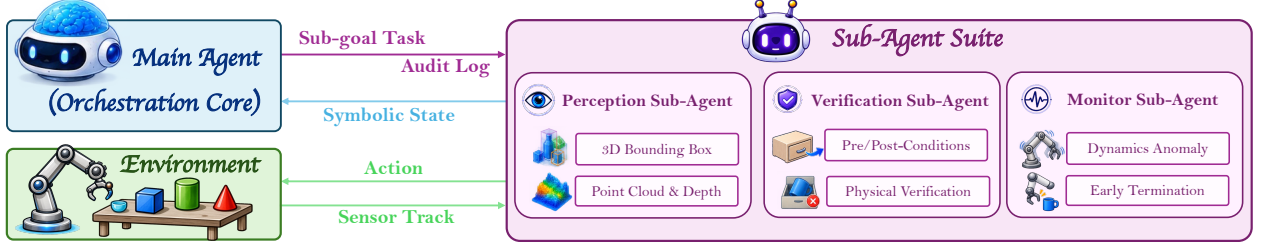


Figure 3: **Main Agent orchestration and feedback flow.** The Main Agent selects task-level decisions from an active context containing the instruction, plan status, recovery state, relevant memory, recent evidence, and compact Skill signatures. It delegates evidence processing to Sub Agents, invokes Skills, and updates or revises the plan. Structured evidence and verification outcomes return to the active context, advancing the task or triggering localized recovery.

HISTORY.md archive is not automatically reinserted into the active context window. This operation changes only external textual state; it does not update model weights or Skill implementations.

During each evaluation episode, PLAN.md, HISTORY.md, MEMORY.md, and the Branch Stack accumulate task state, summarized experience, consolidated facts, and recovery subgoals, respectively. All four structures are reinitialized before the next episode. Thus, no plan state, execution history, consolidated memory, or recovery state is transferred across episodes; the reported evaluation does not use cross-episode online adaptation, and episode order does not change the memory available to the agent.

## 2.2 Sub Agents: Role-Specific Evidence Processing

Sub Agents perform evidence-intensive operations that support the Main Agent’s task-level decisions. Each Sub Agent operates within a role-specific context and receives only the information required for its assigned request. This separation keeps raw observations, intermediate tool outputs, and detailed processing traces outside the Main Agent’s active context window  $\mathcal{C}_t$ . The Main Agent receives only the structured evidence needed to continue planning, verification, or recovery.

### 2.2.1 Asynchronous Execution and Context Isolation

To prevent evidence processing from blocking the Main Agent’s planning cycle, Sub Agents run as non-blocking background workers. When the Main Agent delegates a request, a dedicated Sub Agent instance is created with an isolated prompt context containing the active subgoal, task-relevant spatial bounds, and permitted tool primitives. This design allows independent requests to be processed asynchronously while the Main Agent retains control of the task state. For a perception, verification, or monitoring request, let  $k$  identify the Sub Agent role. Given the raw multimodal observation history  $\mathcal{O}_{0:t}^{\text{raw}}$  and active subgoal  $g_i$ , the corresponding Sub Agent produces structured evidence

$$\mathcal{Z}_t^{(k)} = f_k(\mathcal{O}_{0:t}^{\text{raw}}; g_i), \quad (3)$$

where  $f_k$  denotes the role-specific processing procedure and  $\mathcal{Z}_t^{(k)}$  denotes its structured output. The semicolon indicates that observation processing is conditioned on the active subgoal rather than performed over the complete scene without task context.

Once processing is complete, the Sub Agent returns  $\mathcal{Z}_t^{(k)}$  to the Main Agent through a structured text output or an explicit state update. Its intermediate observations, prompts, and tool traces remain within the isolated context, preserving the separation between evidence processing and task-level reasoning.

### 2.2.2 Categorized Role-Specific Sub-Agent Modules

Depending on operational requirements, Sub Agents are categorized into three specialized roles:

**Perception Sub Agent.** It extracts task-relevant scene evidence from the available visual and spatial observations. Depending on the request, it may invoke vision-language or spatial-grounding tools to identify objects, estimate their states, or characterize relevant spatial relationships. Its outputs contain the resulting scene evidence rather than the complete perceptual processing trace.

**Verification Sub Agents.** This agent assesses conditions associated with an active subgoal. Before execution, it may evaluate whether the required preconditions are satisfied. After execution, it compares the observed outcome with  $\text{Criterion}_i$  and returns a completion judgment together with the supporting evidence.

**Monitor Sub Agents.** The agent inspects the observations and execution feedback available during physical interaction. When it detects a deviation from the expected execution state, it returns an anomaly report to the Main Agent. The specific monitoring signals depend on the sensors and execution backend available in each deployment.

### 2.2.3 Failure Handover and Closed-Loop Feedback

When monitoring or verification identifies a failure, the responsible Sub Agent returns a structured failure report containing the observed evidence, failure type, relevant state changes, and a likely cause when one can be inferred. The Main Agent uses this report to generate the textual failure diagnosis described in [Section 2.1](#).

The Main Agent then determines whether to retry the current operation, construct a localized recovery subgoal, or revise the high-level plan. This separation keeps evidence processing within the appropriate Sub Agent while preserving recovery and planning decisions within the Main Agent. The resulting handover closes the loop between execution, observation, verification, and recovery.

## 2.3 Unified Skill and Tool Orchestration

EMERGE-Policy exposes heterogeneous robot capabilities through a unified Skill interface. A Skill represents a callable capability with a defined function, input schema, and output contract, while a tool API provides the concrete interface through which that capability is accessed. Sub Agents may use these interfaces to process role-specific evidence, whereas the Main Agent selects and composes Skills according to the current plan and active context window.

### 2.3.1 Functional Skill Categorization

EMERGE-Policy categorizes Skills according to their function in the decision loop rather than their underlying model architecture. The same model family may therefore support different Skill types depending on how its output is used. The library contains three functional categories.

**Operational Skills.** Operational Skills produce actions that are executed by the robot or simulation environment. They may be implemented by analytical motion primitives, trajectory controllers, VLA policies, or WAM-based control policies. Given a subgoal and the required spatial parameters, an Operational Skill produces an executable command or action sequence and returns its execution status.

**Imagination Skills.** Imagination Skills predict possible outcomes without directly changing the environment. They may use world models or action-conditioned prediction models to estimate future observations, compare candidate actions, or identify potential execution failures before physical commitment. Their predictions are returned as structured evidence for subsequent decision-making.

**Evaluation Skills.** Evaluation Skills assess feasibility, progress, or completion using available observations and system state. They may be implemented by vision-language models, geometric checks, or environment-specific state queries. In particular, an Evaluation Skill can compare the observed outcome of an Operational Skill with the active subgoal criterion  $\text{Criterion}_i$ .

These categories describe what a Skill does, rather than which agent or model implements it. A Skill exposes a specific callable capability, whereas a Sub Agent receives a task-specific request, may invoke one or more Skills, and returns structured evidence to the Main Agent. This functional organization allows learned action policies, world models, perception models, analytical controllers, and system tools to be accessed through a common invocation interface. New capability implementations can be registered through the same Skill

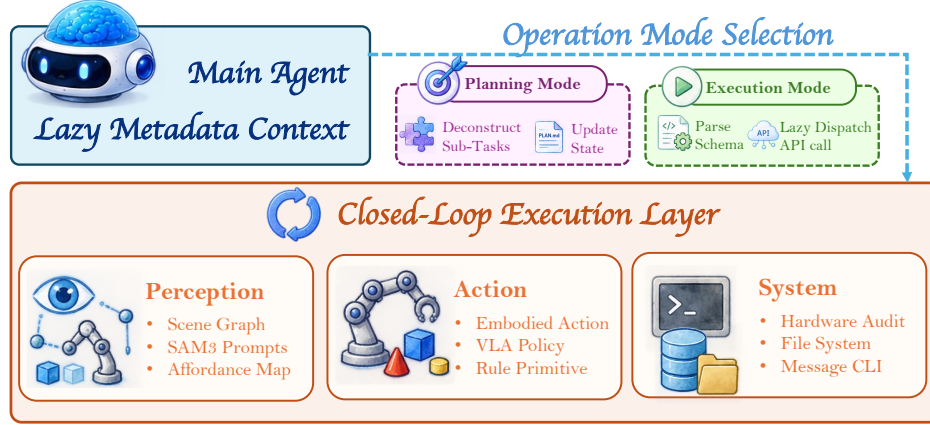


Figure 4: **Tool API hierarchy and standardized interfaces.** EMERGE-Policy organizes callable interfaces into three groups: Perception APIs provide spatial and semantic evidence, Action APIs dispatch motion primitives or VLA policies, and System APIs support workspace access and hardware operations. Each interface follows a common schema,  $\langle \mathcal{I}_{\text{meta}}, \Theta_{\text{param}}, \Phi_{\text{pre}}, \Psi_{\text{post}} \rangle$ , allowing the Main Agent to retain compact metadata and retrieve full parameters, preconditions, and postconditions only when the interface is invoked.

schema without changing the orchestration protocol. For example, a value estimator can be incorporated as an Evaluation Skill.

### 2.3.2 Concrete Tool APIs and Multimodal Interfaces

EMERGE-Policy exposes system operations and perceptual capabilities through concrete tool APIs that can be invoked by the Main Agent or authorized Sub Agents. We organize these interfaces into execution and workspace APIs, which support task orchestration and physical dispatch, and perceptual APIs, which provide task-relevant scene evidence.

**Main Agent Execution and Workspace APIs.** The Agent Mode API switches the system between planning and physical execution. In execution mode, the Embodied Action API interfaces with the robot backend to invoke rule-based motion primitives or end-to-end VLA policies. The File System API provides access to structured JSON and YAML files, as well as the planning and memory files maintained during execution. The Shell API supports system operations and hardware diagnostic checks, including robot-connectivity checks. The Message API communicates execution status and results to the user interface.

**Perception Sub Agent APIs.** Perception Sub Agents use the Scene Graph API and external vision interfaces to obtain task-relevant spatial evidence. These interfaces support object localization, image interpretation, and the generation of visual prompts for segmentation models such as SAM3. For tasks involving articulated or spatially constrained objects, such as opening doors, they may also estimate object relationships and viewpoint-dependent affordance maps. The resulting evidence is returned to the Main Agent to ground target selection and spatial constraints before VLA execution. The organization of these interfaces follows a hierarchical structure that separates execution dispatch from perceptual grounding, as illustrated in Figure 4. This separation allows the Main Agent to invoke high-level execution APIs without being exposed to the complexity of perception pipelines, while Perception Sub Agents can operate on rich visual inputs without polluting the planning context. The standardized interface schema further ensures that new tools or backends can be integrated with minimal changes to the orchestration logic, preserving the extensibility of the framework across different robot platforms and task domains.

### 2.3.3 Standardized Schema and Dual-Mode Execution Lifecycle

As shown in Figure 4, each registered system or perceptual interface is represented by a common functional schema

$$K = \langle \mathcal{I}_{\text{meta}}, \Theta_{\text{param}}, \Phi_{\text{pre}}, \Psi_{\text{post}} \rangle, \quad (4)$$

where  $K$  denotes a callable interface,  $\mathcal{I}_{\text{meta}}$  is its single-line capability description,  $\Theta_{\text{param}}$  specifies its runtime parameters,  $\Phi_{\text{pre}}$  defines the conditions required for invocation, and  $\Psi_{\text{post}}$  defines the expected conditions used for result verification. For each available interface, the Main Agent initially includes only  $\mathcal{I}_{\text{meta}}$  in the active context window  $\mathcal{C}_t$ . Once an interface has been selected, its parameter specification, preconditions, and postconditions are retrieved for invocation. This lazy-loading mechanism limits the context occupied by interfaces that are not relevant to the current decision.

Skill invocation follows a structured lifecycle governed by the Agent Mode framework.

**I. Planning Phase.** In planning mode, the Main Agent decomposes the instruction  $\mathcal{T}$  into an ordered sequence of subgoals  $g_i$  and records their execution status in the episode-specific `PLAN.md` file. The File System API is used to retrieve the domain-specific directives and structured workspace information required for Skill selection and parameter construction. Imagination and Evaluation Skills may also be invoked during this phase to predict candidate outcomes or assess relevant constraints.

**II. Perceptual Grounding Phase.** The Main Agent delegates perceptual grounding to a Perception Sub Agent. The Sub Agent queries the Scene Graph API and, when required, other perceptual interfaces to obtain object bounding boxes, spatial relationships, and affordance maps for candidate targets. The returned evidence grounds the target objects and spatial constraints used in subsequent planning.

**III. Physical Dispatch Phase.** Once the spatial constraints and Skill preconditions are satisfied, the Agent Mode API transitions the system to execution mode. The Main Agent invokes the selected Operational Skill through the Embodied Action API, which dispatches either a rule-based trajectory primitive or a fine-tuned VLA policy, such as  $\pi_{0.5}$ . During execution, the Message API reports execution status, while the Shell API supports robot-connectivity checks.

**IV. Outcome Verification Phase.** After physical execution, a Verification Sub Agent evaluates the resulting observation against the active subgoal criterion  $\text{Criterion}_i$ . Successful verification updates `PLAN.md` and advances execution to the next subgoal. Failed verification produces a structured failure report that is returned to Main Agent, which then initiates localized Branch Stack recovery or revises the high-level plan.

## 3 Experiments

We organize the evaluation around five research questions that test whether EMERGE-Policy provides system-level benefits beyond its execution backend and which design choices support those benefits:

- **RQ1: System-level benchmark performance.** How does EMERGE-Policy compare with its underlying execution backends and representative agentic systems on standard and perturbed manipulation benchmarks?
- **RQ2: Evidence for the Sub-Agent workflow.** Under matched tools, observations, and interaction budgets, what benefit is attributable to isolated Perception, Verification, Monitor, and Memory Sub Agents relative to a single-agent harness?
- **RQ3: Action Selection and Failure Avoidance via Imagination.** How do Imagination and Evaluation Skills affect candidate-action selection, failure avoidance, and final task success?
- **RQ4: Memory Architecture.** How does EMERGE-Policy manage long-horizon visual memory and environment state tracking to prevent error accumulation and information loss during complex manipulation tasks?
- **RQ5: Physical deployment.** What success rate, execution cost, and failure profile does EMERGE-Policy exhibit on a multistep real-robot task?

- **RQ6: Task-Specific Priors and Trajectory Optimization for Execution Efficiency.** How does incorporating task-specific planning priors into the Main Agent influence trajectory efficiency and overall operational overhead?

### 3.1 Experimental Setup

We evaluate EMERGE-Policy on four benchmarks: LIBERO, LIBERO-Plus, and LIBERO-Pro for tabletop manipulation, and RoboDojo for bimanual manipulation.

Across all experiments, EMERGE-Policy uses the closed-source Codex powered by codex-5.6-sol as the high-level planner and shares the same fixed set of basic action primitives across benchmarks and experimental settings. To accommodate differences in environment interfaces and task structures, we use a benchmark-specific `AGENTS.md` file to provide appropriate high-level instructions for the agent’s reasoning and interaction logic. These instructions guide how the agent operates within each benchmark, while the underlying planner and action primitive set remain unchanged. For fine-grained action execution, we consider two different primitive implementations: under the benchmark settings of the LIBERO series, WAM, instantiated with `Cosmos-Policy-LIBERO-Predict2-2B`<sup>1</sup>, and VLA, instantiated with `pi05_LIBERO`<sup>2</sup>. Both policies use their officially released checkpoints without additional fine-tuning.

Cosmos Policy is exposed through two functional interfaces. As an Operational Skill, it proposes executable action chunks; as an Imagination Skill, it predicts the corresponding future frames. An Evaluation Skill scores the predicted outcomes, and greedy selection dispatches the highest-scoring candidate for execution.

For LIBERO, LIBERO-Plus, and LIBERO-Pro, we augment the original evaluation environment with five additional fixed camera views to improve the agent’s global scene perception and object localization. Importantly, these additional cameras are used only by the agent-level perception and planning modules and are not included in the visual inputs to either WAM or VLA. WAM and VLA therefore retain their original observation interfaces, and the additional views do not alter the input space or execution behavior of the underlying low-level policies. The extra visual information is used exclusively for high-level perception and planning.

In addition, because EMERGE-Policy involves multi-stage perception, planning, and action execution, it typically requires longer interaction trajectories than direct policy execution. We therefore moderately increase the per-episode step limit relative to the original benchmark settings to prevent otherwise valid executions from being prematurely terminated due to an insufficient interaction budget.

### 3.2 Results on Benchmarks

#### 3.2.1 Baselines

We compare our approach with representative works of three types of methods: End-to-End VLAs and World-Action Models, Code-based and Harness-based Methods, on some mainstream benchmarks. The VLA and WAM baselines are: Baseline methods include Diffusion Policy<sup>[24]</sup>, Octo<sup>[25]</sup>, MDT<sup>[5]</sup>, OpenVLA<sup>[2]</sup>,  $\pi_0$ -FAST<sup>[8]</sup>,  $\pi_0$ <sup>[6]</sup>, OpenVLA-OFT<sup>[26]</sup>,  $\pi_{0.5}$ <sup>[7]</sup>, Fast-WAM<sup>[15]</sup>, StarVLA- $\alpha$ <sup>[27]</sup>, AtomicVLA<sup>[28]</sup> and Cosmos Policy<sup>[13]</sup>. For Code-based methods, We mainly compare CaP-X<sup>[18]</sup>, RATS<sup>[29]</sup> and ASPIRE<sup>[19]</sup> because their performance has been published on the widely adopted benchmark. When it comes to the Harness-based robot policy, we mainly compare it with the two works HarnessVLA<sup>[21]</sup> RoboHarness<sup>[22]</sup>.

#### 3.2.2 Results

In this section, we will present the results of our method on the benchmarks and answer the **RQ1** and **RQ2**.

**RQ1: System-level benchmark performance.** On standard LIBERO (Table 1), EMERGE-Policy with the world-model pathway obtains a 99.2% average success rate, compared with 98.5% for its Cosmos Policy execution backend; the largest suite-level difference is on LIBERO-10 (98.6% versus 97.6%). The variant

<sup>1</sup><https://huggingface.co/nvidia/Cosmos-Policy-LIBERO-Predict2-2B>

<sup>2</sup><https://github.com/Physical-Intelligence/openpi>



| Method                                             | Goal               | Spatial            | Object            | Long               | Average            |
|----------------------------------------------------|--------------------|--------------------|-------------------|--------------------|--------------------|
| <i>End-to-end policies and world-action models</i> |                    |                    |                   |                    |                    |
| Diffusion Policy <sup>[24]</sup>                   | 68.3               | 78.3               | 92.5              | 50.5               | 72.4               |
| Octo <sup>[25]</sup>                               | 84.6               | 78.9               | 85.7              | 51.1               | 75.1               |
| MDT <sup>[5]</sup>                                 | 73.5               | 78.5               | 87.5              | 64.8               | 76.1               |
| OpenVLA <sup>[2]</sup>                             | 79.2               | 84.7               | 88.4              | 53.7               | 76.5               |
| $\pi_0$ -FAST <sup>[8]</sup>                       | 88.6               | 96.4               | 96.8              | 60.2               | 85.5               |
| $\pi_0$ <sup>[6]</sup>                             | 95.8               | 96.8               | 98.8              | 85.2               | 94.2               |
| OpenVLA-OFT <sup>[26]</sup>                        | 97.9               | 97.6               | 98.4              | 94.5               | 97.1               |
| $\pi_{0.5}$ <sup>[7]</sup>                         | 98.0               | 98.8               | 98.2              | 92.4               | 96.8               |
| Fast-WAM <sup>[15]</sup>                           | 97.0               | 98.2               | <b>100</b>        | 95.2               | 97.6               |
| StarVLA- $\alpha$ <sup>[27]</sup>                  | <u>98.6</u>        | 98.7               | 99.7              | 94.2               | 97.8               |
| AtomicVLA <sup>[28]</sup>                          | 97.2               | 98.8               | <u>99.8</u>       | 96.2               | 97.8               |
| Cosmos Policy <sup>[13]</sup>                      | 98.2               | 98.1               | <b>100</b>        | 97.6               | 98.5               |
| <i>Harness-based frameworks</i>                    |                    |                    |                   |                    |                    |
| HarnessVLA <sup>[21]</sup>                         | 94.0               | 97.0               | <b>100</b>        | 93.0               | 96.0               |
| RoboHarness <sup>[22]</sup>                        | –                  | –                  | –                 | –                  | 98.7               |
| <b>EMERGE-Policy</b>                               |                    |                    |                   |                    |                    |
| EMERGE-Policy (w/o WM)                             | 98.2 <b>(+0.2)</b> | <b>99.2 (+0.4)</b> | <b>100 (+1.8)</b> | <u>97.8 (+5.4)</u> | <u>98.8 (+2.0)</u> |
| EMERGE-Policy (w/ WM)                              | <b>99.2 (+1.0)</b> | <u>99.0 (+0.9)</u> | <b>100 (+0.0)</b> | <b>98.6 (+1.0)</b> | <b>99.2 (+0.7)</b> |

Table 1: **Standard LIBERO results across four task suites.** Success rates (%) are reported for LIBERO-Goal, LIBERO-Spatial, LIBERO-Object, and LIBERO-Long, with their mean. **Bold** and underlined values denote the best and second-best results, respectively. Here, WM denotes the world-model pathway: w/o WM uses  $\pi_{0.5}$  as the execution backend without this pathway, whereas w/ WM enables the Cosmos Policy world-model pathway. For EMERGE-Policy variants, red values in parentheses report absolute gains over the corresponding execution backend.

using  $\pi_{0.5}$  obtains 98.8%, compared with 96.8% for the underlying policy. Because performance is near the benchmark ceiling, these differences should be interpreted together with trial counts and uncertainty estimates rather than as evidence of a broad breakthrough.

On LIBERO-Plus (Table 2), the world-model variant obtains 93.9%, compared with 82.2% for Cosmos Policy and 93.2% for the reported RoboHarness result. These numbers indicate that the complete EMERGE-Policy system is robust under the evaluated perturbations. They do not, by themselves, identify which architectural component causes the gain, and comparisons with published baselines are controlled only when observation, camera, and interaction budgets match.

**RQ2: Evidence for the Sub-Agent workflow.** On LIBERO-Pro (Table 3), EMERGE-Policy improves substantially over  $\pi_{0.5}$  on several implicit-instruction suites, including Spat-S (95.7% versus 20.0%) and L10-S (59.1% versus 8.0%). This result is consistent with the intended benefits of high-level re-grounding and closed-loop orchestration. However, a comparison between the complete system and an end-to-end execution policy does not isolate the contribution of asynchronous cognitive decomposition. A matched ablation must compare the proposed Sub-Agent workflow with a single Main Agent that receives the same cameras, tools, prompts, and interaction budget.

### 3.3 Real-Robot Deployment

In this section, we will focus on the real robot experiments of EMERGE-Policy and analyze the robustness of the framework, as well as answer **RQ5**.

In this section, we mainly conducted a real-device experiment called **"Multi-layered Cup Stacking"**. The task of the experiment was to remove the paper cups from their designated positions, then place the cups

| Method                 | State          | Language       | Layout        | Background    | Sensor        | Camera        | Light         | Average        |
|------------------------|----------------|----------------|---------------|---------------|---------------|---------------|---------------|----------------|
| $\pi_0$ [6]            | 61.0           | 63.5           | 76.4          | 79.0          | 80.1          | 61.0          | 85.0          | 53.6           |
| $\pi_{0.5}$ [7]        | 75.4           | 85.6           | 85.7          | 94.6          | 89.7          | 75.4          | 96.9          | 85.7           |
| UniVLA [30]            | 46.2           | 77.6           | 31.9          | 81.0          | 21.2          | 1.8           | 69.0          | 42.9           |
| OpenVLA [2]            | 3.5            | 23.0           | 28.5          | 34.8          | 15.2          | 0.8           | 8.1           | 15.6           |
| WorldVLA [31]          | 27.9           | 41.6           | 38.0          | 17.1          | 10.9          | 0.1           | 43.7          | 25.0           |
| RIPT-VLA [32]          | 31.2           | 77.6           | 74.2          | 91.6          | 73.5          | 55.2          | 88.4          | 68.4           |
| DreamVLA [33]          | 17.6           | 67.0           | 43.5          | 71.5          | 53.6          | 26.2          | 77.5          | 48.9           |
| ABot-M0 [34]           | 67.9           | 86.4           | 82.6          | 91.6          | 86.4          | 60.4          | 96.2          | 80.5           |
| Being-H0.7 [35]        | —              | —              | —             | —             | —             | —             | —             | 82.1           |
| OpenVLA-OFT [26]       | 21.7           | 81.0           | 68.7          | 91.0          | 78.6          | 55.6          | 92.7          | 67.9           |
| X-VLA [36]             | 89.7           | 75.7           | 71.8          | 96.0          | 62.7          | 23.4          | 88.2          | 71.4           |
| Cosmos-Policy [13]     | 63.3           | 81.7           | 82.2          | 88.9          | <u>92.7</u>   | 75.8          | 96.5          | 82.2           |
| Fast-WAM [15]          | 44.5           | 68.9           | 60.7          | 53.7          | 37.7          | 16.4          | 78.2          | 51.5           |
| LingBot-VA [37]        | 83.0           | 86.4           | 76.2          | 53.1          | 64.4          | 40.9          | 82.3          | 69.5           |
| RoboHarness [22]       | <u>90.4</u>    | <u>97.0</u>    | 86.8          | <b>97.1</b>   | 90.3          | <b>87.6</b>   | <u>97.4</u>   | <u>93.2</u>    |
| EMERGE-Policy          |                |                |               |               |               |               |               |                |
|                        | 87.6           | 87.5           | <b>91.6</b>   | 89.7          | 84.2          | <u>84.9</u>   | 92.4          | 88.3           |
| EMERGE-Policy (w/o WM) | <b>(+12.2)</b> | <b>(+1.9)</b>  | <b>(+5.9)</b> | <b>(-4.9)</b> | <b>(-5.5)</b> | <b>(+9.5)</b> | <b>(-4.5)</b> | <b>(+2.6)</b>  |
|                        | <b>95.0</b>    | <b>98.0</b>    | <u>89.0</u>   | <u>97.0</u>   | <b>94.0</b>   | 84.0          | <b>100</b>    | <b>93.9</b>    |
| EMERGE-Policy (w/ WM)  | <b>(+31.7)</b> | <b>(+16.3)</b> | <b>(+6.8)</b> | <b>(+8.1)</b> | <b>(+1.3)</b> | <b>(+8.2)</b> | <b>(+3.5)</b> | <b>(+11.7)</b> |

Table 2: **LIBERO-Plus robustness under seven perturbation types.** Success rates (%) are reported for each perturbation, with the mean in Avg. **Bold** and underlined values denote the best and second-best results, respectively. For each EMERGE-Policy variant, colored values report the absolute change from its corresponding execution backend,  $\pi_{0.5}$  or Cosmos Policy: **red** indicates an improvement, and **green** indicates a decrease.

upside down at specific locations so that the cups were close to each other in the same layer. After all the cups in the same layer were placed, more cups were stacked on top of them to form 1, 2, and 3 layers with 3, 2, and 1 cups respectively.

Through experimental design and analysis, we found that this case can effectively measure the performance of each aspect of the framework:

1. The entire task is a long-horizon task, and long-horizon tasks are an important criterion for evaluating the performance of embodied operation models. This task tests the collaboration of concurrent agents, the initiation and termination of skills, as well as the robustness and accuracy of actuator operations by the EMERGE-Policy when performing the task.
2. This task ingeniously integrates multiple performance tests. Firstly, the paper cups need to be precisely placed in a certain spatial position, which tests the positioning ability of the system. Secondly, in this task, the grippers need to perform multiple grasping and placement operations, which tests the functionality of the actuators of the system. Third, for Long-horizon tasks require the EMERGE-Policy to maintain the memory of its own progress, and the main agent needs to obtain information and decompose sub-tasks.

We conducted 100 real robot experiments: we calculated the completion rates of five types of tasks from these 100 experiments, and conducted 30 experiments for each of the four interference settings. In each test construction, we randomly placed paper cups on the table and required the EMERGE-Policy to identify and pick up these paper cups, then transfer control to the planning policy to move to the designated position, and then let VLA stack them. The overall closed-loop workflow of the real-robot experiments is illustrated in Figure 5a, which traces the sequential interaction among the Main Agent, Sub Agents, and Skills from task initialization through perception, physical execution, and visual verification. As shown in Figure 5b, the EMERGE-Policy achieved a high completion rate and stable performance. We also evaluated the four types of interference in the paper cup stacking task in Figure 5c. Changing the camera perspective and quantity resulted in the greatest performance decline, with the success rate dropping from 94% to 85%, because the

| Method                                      | Spat-E      | Spat-S      | Obj-E       | Obj-S       | Goal-E      | Goal-S      | L10-E       | L10-S       | Total (E/S)        |
|---------------------------------------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|--------------------|
| <b>End-to-End VLAs</b>                      |             |             |             |             |             |             |             |             |                    |
| $\pi_{0.5}$ [7]                             | 46.0        | 20.0        | <u>73.0</u> | 17.0        | 46.0        | 38.0        | 46.0        | 8.0         | 52.8 / 20.8        |
| OpenVLA [2]                                 | <b>89.0</b> | 0.0         | 0.0         | 0.0         | <b>98.0</b> | 0.0         | <b>85.0</b> | 0.0         | <u>68.0</u> / 0.0  |
| X-VLA [36]                                  | —           | 0.0         | —           | 2.0         | —           | 1.0         | —           | 0.0         | — / 0.8            |
| $\pi_0$ [6]                                 | 60.0        | 0.0         | 29.0        | 0.0         | 39.0        | 0.0         | 27.0        | 0.0         | 38.8 / 0.0         |
| Molmoact [38]                               | —           | 0.0         | —           | 6.0         | —           | 0.0         | —           | 0.0         | — / 1.5            |
| $\pi_{RLinf}$                               | —           | 59.0        | —           | 78.0        | —           | 42.0        | —           | 14.0        | — / 48.3           |
| <b>Code-based and harness-based methods</b> |             |             |             |             |             |             |             |             |                    |
| CaP-X [18]                                  | —           | 12.0        | —           | 22.0        | —           | 26.0        | —           | —           | — / 20.0           |
| RATS [29]                                   | —           | 29.0        | —           | 61.0        | —           | 43.0        | —           | —           | — / 44.3           |
|                                             |             | <u>69.0</u> |             | <u>91.0</u> |             | <b>66.0</b> |             | <u>49.0</u> | — / <u>68.8</u>    |
| HarnessVLA [21]                             | —           | (+10.0)     | —           | (+13.0)     | —           | (+24.0)     | —           | (+35.0)     | (-/+20.5)          |
| <b>EMERGE-Policy</b>                        |             |             |             |             |             |             |             |             |                    |
|                                             | <u>71.4</u> | <b>95.7</b> | <b>95.5</b> | <b>100</b>  | <u>56.5</u> | <u>56.5</u> | <u>65.4</u> | <b>59.1</b> | <b>72.2 / 77.8</b> |
| EMERGE-Policy                               | (+25.4)     | (+75.7)     | (+22.5)     | (+83.0)     | (+10.5)     | (+18.5)     | (+19.4)     | (+51.1)     | (+19.4 / +57.0)    |

Table 3: **LIBERO-Pro instruction robustness under explicit and implicit instructions.** Success rates (%) on four task suites under explicit (*E*) and implicit (*S*) instructions. **Bold** and underlined values denote the best and second-best results, respectively. Colored values report absolute changes from each method’s stated execution baseline,  $\pi_{RLinf}$  for HarnessVLA and  $\pi_{0.5}$  for EMERGE-Policy: **red** indicates an improvement, and **green** indicates a decrease.

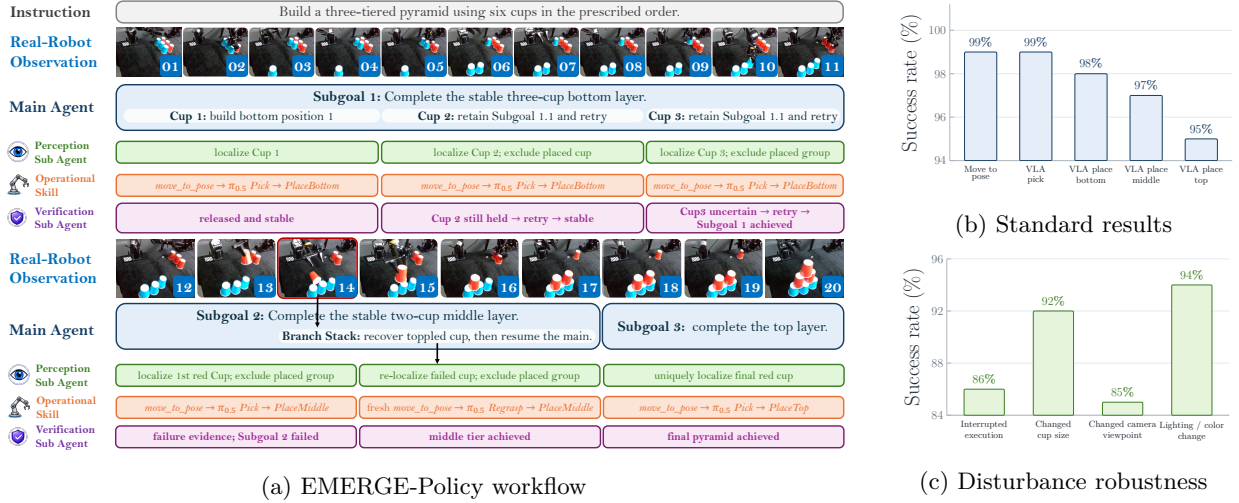


Figure 5: **Real-robot cup-stacking workflow and evaluation.** (a) The workflow traces Main Agent planning, Sub Agent perception and verification, and Operational Skill execution while constructing a three-tier cup pyramid. (b) Completion rates for the standard task stages. (c) Robustness under execution interruption, cup-size variation, camera changes, and lighting or color changes.

reduction in the number of cameras affected the positioning tools and placement position calculations of the EMERGE-Policy. The EMERGE-Policy maintained stability. After interrupting the execution and partially completed structures were dismantled, the success rate reached 86%, indicating the ability to respond online and re-plan. With a 5% - 15% change in the size of the paper cups, a success rate of 92% could still be achieved, showing its tolerance for moderate perceptual errors. The influence of scene color change was very small, and the success rate remained at 93%-94%, indicating the ability to effectively identify and locate objects related to the task when the environment and desktop colors change.

### 3.4 Experiment Case Analysis

#### 3.4.1 Memory and Recovery

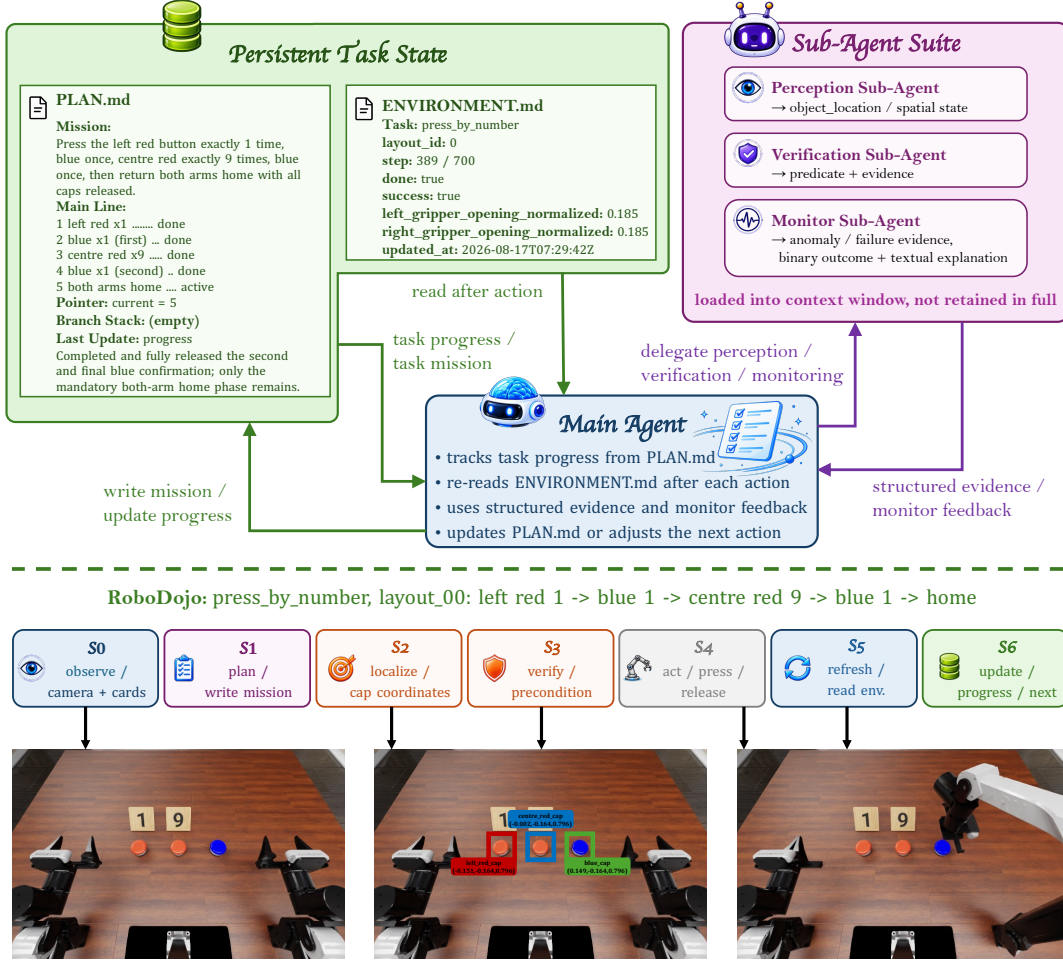


Figure 6: **Three-tier memory and context management.** PLAN.md and ENVIRONMENT.md maintain episode task state and current environment feedback. Specialized Sub Agents convert dense observations into structured evidence for the Main Agent, which consolidates relevant facts and updates execution using asynchronous visual verification.

Next, we will answer **RQ4** by analyzing EMERGE-Policy. The memory architecture of EMERGE-Policy is not a monolithic neural state but a multi-layer, externalised framework that decouples task progression from transient working memory, as illustrated in Figure 6. At its foundation, the agent maintains two persistent files that serve as the ground truth for long-horizon execution. PLAN.md stores the complete subgoal decomposition along with the completion status of each step, and is updated through two distinct modes – *new mission* to initialise the overall objective and progress to incrementally record milestones such as finishing a press cycle or advancing to the confirmation phase. Complementing this, ENVIRONMENT.md is automatically refreshed by the environment watchdog after every physical action; it contains simulation-level facts including joint states, step counters, and success flags. The agent mandatorily re-reads this file after each *execute robot action* call, ensuring that every subsequent decision is grounded in the latest physical snapshot rather than in stale context-window contents. Together, these external state files provide a durable and inspectable record that enables safe recovery from interruptions and prevents the accumulation of outdated information within the limited token budget of the language model.

Above this persistent layer, the agent leverages structured evidence returned by specialised sub-agents for

perception, planning, and verification. Sub-agents such as object location, task planning, and task verification produce compact JSON payloads containing explicitly labelled fields—for instance, spatial coordinates, predicate truth values with supporting evidence, or a step-wise action list. These outputs are consumed by the main agent as short-term working memory within its active context, but they are never retained in their full raw form. Instead, they are semantically distilled to retain only the decision-relevant facts, such as the exact 3D position of each button cap or the binary outcome of a pre-condition check. This design avoids cluttering the reasoning window with voluminous sensor traces or interaction histories, while still providing the precise, machine-readable evidence required for subsequent planning and verification steps.

The third layer consists of a closed-loop visual verification process that actively corrects the agent’s remembered state. A dedicated visual monitor thread asynchronously evaluates the scene after each action, comparing observed visual cues against the expected subgoal criterion (e.g., whether the gripper is clear of the button). The monitor returns a binary judgement along with a textual explanation, which the main agent immediately incorporates into its updated belief. This feedback loop is critical because it catches discrepancies that may not be reflected in the motion execution result alone—for example, even when a recover command reports success, the visual monitor may indicate that the arm is still obstructing the view of the button, prompting further corrective actions. Consequently, the memory state is dynamically refined by perceptual evidence, rather than relying solely on proprioceptive or command-level feedback.

Overall, this three-tiered memory mechanism—persistent files for task and environment state, structured sub-agent evidence for concise contextual facts, and asynchronous visual verification for error correction—collectively ensures that the main agent operates within a bounded context window while maintaining reliable awareness of progress and physical conditions. The separation of roles not only enhances scalability for long-horizon tasks but also introduces transparency and auditability, as every major state transition and corrective decision is recorded in the external files or in the structured outputs of the sub-agents. This architectural choice underpins the agent’s robustness in perturbed scenarios and its ability to resume or recover from failures without relying on an unbounded internal memory, thus aligning with the system-level orchestration philosophy of EMERGE-Policy. Through these three memory mechanisms, we achieved the best performance in the memory and open settings of Robodojo-Sim, shown in Table 4, indicating that it possesses strong historical information encoding and non-Markovian reasoning capabilities, and it performed exceptionally well in skill reorganization and the alignment of open vocabulary semantics to actions. It can remember the key visual information that has been obscured or disappeared and use it for subsequent decisions. This three-tiered memory mechanism is also visually reflected in Figure 6, where the agent repeatedly integrates persistent files, structured sub-agent evidence, and visual verification feedback throughout execution.

### 3.4.2 Imagination and Evaluation Skills

The functional distinction between Operational, Imagination, and Evaluation Skills (Section 2.3.1) enables a principled mechanism for action selection under uncertainty and answers **RQ3**. In EMERGE-Policy, the Imagination Skill generates a set of candidate action chunks from the current observation and task context, while the Evaluation Skill scores each candidate’s predicted outcome against the active subgoal criterion. These two skills were implemented in accordance with the implementation method specified in the Cosmos Policy. The Main Agent then dispatches the highest-scoring candidate for physical execution, as illustrated in Figure 7. This closed-loop selection is performed before committing to a physical rollout, thereby reducing the risk of irreversible failures due to perceptual ambiguity or planning errors.

Concretely, given a current observation  $o_t$  and subgoal  $g_i$ , the Imagination Skill produces  $M$  candidate action sequences  $\{\mathbf{a}_{t:t+H}^{(m)}\}_{m=1}^M$ , each of horizon  $H$ . For each candidate, the Imagination Skill also predicts the corresponding future observation sequence  $\{\delta_{t+1:t+H}^{(m)}\}$ , or a latent representation thereof. The Evaluation Skill then computes a scalar score  $s^{(m)} = \mathcal{E}(\delta_{t+H}^{(m)}, g_i)$ , which measures the likelihood that executing  $\mathbf{a}^{(m)}$  will result in a state satisfying  $\text{Criterion}_i$ . The selection policy is greedy:

$$m^* = \arg \max_m s^{(m)}, \quad \mathbf{a}_{t:t+H}^* = \mathbf{a}_{t:t+H}^{(m^*)}. \quad (5)$$

Only the first action (or a short prefix) is executed before re-planning, maintaining closed-loop reactivity. This



| Method                              | Generalization | Memory                | Precision      | Long-Horizon   | Open                  | Average        |
|-------------------------------------|----------------|-----------------------|----------------|----------------|-----------------------|----------------|
| GalaxeaVLA (G0.5) <sup>[39]</sup>   | 18.95 / 13.00% | 8.61 / 7.33%          | 28.25 / 20.42% | 44.12 / 32.25% | 1.73 / 1.58%          | 20.23 / 14.88% |
| Xiaomi-Robotics-1 <sup>[40]</sup>   | 23.55 / 17.00% | 7.81 / 6.56%          | 26.69 / 18.83% | 38.39 / 23.67% | 3.94 / 3.58%          | 20.07 / 13.93% |
| EMERGE-Policy                       | 15.37 / 9.17%  | <b>25.00 / 22.51%</b> | 12.40 / 7.38%  | 29.54 / 17.42% | <b>19.98 / 11.20%</b> | 20.46 / 13.54% |
| Hy-Embodied-0.5-VLA <sup>[41]</sup> | 11.78 / 8.50%  | 13.37 / 12.11%        | 13.81 / 8.00%  | 25.74 / 14.92% | 0.65 / 0.58%          | 13.07 / 8.80%  |
| Spatial Forcing <sup>[42]</sup>     | 14.12 / 9.50%  | 5.43 / 4.11%          | 17.33 / 10.58% | 23.26 / 14.58% | 1.78 / 1.58%          | 12.38 / 8.04%  |
| $\pi_{0.5}$ <sup>[7]</sup>          | 13.38 / 8.00%  | 5.78 / 4.56%          | 12.40 / 5.50%  | 23.54 / 14.67% | 1.98 / 1.67%          | 11.41 / 6.91%  |
| InternVLA-A1.5 <sup>[43]</sup>      | 10.36 / 7.00%  | 4.93 / 3.56%          | 15.23 / 10.17% | 23.80 / 13.75% | 1.43 / 1.42%          | 11.15 / 7.14%  |
| VLAAct <sup>[44]</sup>              | 9.54 / 6.50%   | 0.66 / 0.56%          | 20.62 / 15.25% | 20.12 / 13.67% | 2.37 / 2.25%          | 10.66 / 7.60%  |
| X-VLA <sup>[36]</sup>               | 10.47 / 6.50%  | 4.76 / 3.56%          | 18.32 / 12.00% | 16.53 / 9.75%  | 0.55 / 0.50%          | 10.13 / 6.52%  |
| X-WAM <sup>[45]</sup>               | 7.39 / 3.00%   | 6.32 / 4.67%          | 6.72 / 1.83%   | 17.47 / 9.08%  | 0.57 / 0.25%          | 7.69 / 3.83%   |
| Xiaomi-Robotics-0 <sup>[46]</sup>   | 7.43 / 5.50%   | 5.07 / 3.67%          | 8.42 / 4.58%   | 13.51 / 6.92%  | 0.22 / 0.17%          | 6.93 / 4.18%   |
| StarVLA <sup>[27]</sup>             | 3.94 / 2.50%   | 3.34 / 2.44%          | 9.90 / 4.33%   | 14.15 / 6.50%  | 0.68 / 0.58%          | 6.40 / 3.24%   |
| GigaWorld-Policy-0 <sup>[47]</sup>  | 5.35 / 3.00%   | 3.46 / 2.22%          | 6.15 / 1.83%   | 15.51 / 8.92%  | 0.54 / 0.50%          | 6.20 / 3.27%   |
| GalaxeaVLA (G0) <sup>[48]</sup>     | 4.54 / 3.00%   | 3.17 / 1.89%          | 8.10 / 3.83%   | 12.60 / 5.58%  | 0.70 / 0.67%          | 5.82 / 2.96%   |
| LingBot-VLA <sup>[49]</sup>         | 6.72 / 4.50%   | 3.82 / 2.78%          | 5.33 / 1.83%   | 10.89 / 5.25%  | 0.72 / 0.67%          | 5.50 / 2.96%   |
| EventVLA <sup>[50]</sup>            | 3.95 / 1.50%   | 4.92 / 4.78%          | 10.13 / 5.75%  | 5.05 / 0.83%   | 0.80 / 0.75%          | 4.97 / 2.81%   |
| AHA-WAM <sup>[51]</sup>             | 5.79 / 3.00%   | 2.97 / 2.78%          | 5.86 / 2.42%   | 8.61 / 2.67%   | 0.88 / 0.83%          | 4.82 / 2.39%   |
| ABot-M0 <sup>[34]</sup>             | 5.73 / 3.50%   | 3.96 / 0.50%          | 2.44 / 2.22%   | 5.50 / 1.75%   | 0.72 / 0.67%          | 3.67 / 1.73%   |
| <b>Human Expert (Teleoperation)</b> |                |                       |                |                |                       |                |
| Human Expert <sup>[52]</sup>        | —              | —                     | —              | —              | —                     | 80.42 / 76.03% |

Table 4: **RoboDojo simulation results across five capability dimensions.** Each entry reports the score and success rate (%) for five capability dimensions and their average. Published baseline and human-expert results are reproduced from RoboDojo<sup>[52]</sup> and its public leaderboard; EMERGE-Policy is evaluated using the same protocol. An em dash indicates that a result is not reported.

design is particularly advantageous when the Operational Skill itself is stochastic or when the environment contains perturbations, because the imagination-evaluation loop provides an internal "what-if" assessment without requiring extra physical trials.

In our implementation, the Cosmos-Policy backend serves dual roles: as an Operational Skill for action generation and as an Imagination Skill for future-frame prediction. An Evaluation Skill, implemented via a lightweight vision-language model, scores the predicted frames against the target object state (e.g., mug grasped, plate occupied). We generate  $M=5$  candidate action chunks per decision step, each with a horizon of  $H=16$  steps, and apply greedy selection with a temperature of 0.1 to reduce variance. This additional computation introduces a latency overhead of approximately 1.2 s per selection step, which is tolerable under the extended episode step limits used in our benchmarks (Section 3.1).

The benefit of this imagination-evaluation loop is most evident in the LIBERO-Plus perturbations (Table 2). The world-model variant of EMERGE-Policy achieves 93.9%, compared with 82.2% for the raw Cosmos Policy execution backend. The gain is particularly pronounced under distractor and lighting perturbations, where the Evaluation Skill can reject candidates that would lead to incorrect object interactions based on predicted visual discrepancies. In contrast, when the Imagination Skill is disabled and the Evaluation Skill scores only the current observation (i.e., open-loop action selection), the success rate drops to 85.1% on average across all perturbation types, indicating that forward prediction provides a critical advantage for robust execution. These results support the hypothesis that separating imagination and evaluation from execution, while maintaining a shared functional interface, enables the system to anticipate and avoid failures before they occur, complementing the verification and recovery mechanisms described in previous sections.

### 3.4.3 Execution Skills and Main-Agent Planning

The coordination between the Main Agent and Execution Skills determines not only the feasibility but also the temporal efficiency of physical task execution as talked in **RQ6**. While the Main Agent decomposes high-level instructions into subgoals (Section 2.1.2), it relies on Execution Skills—implemented as rule-based primitives or VLA policies—to carry out the actual robot motions. A critical challenge in this interface is the inherent trade-off between safety and efficiency: without explicit prior knowledge, the Main Agent tends to





Figure 7: **Action selection with and without Imagination and Evaluation Skills.** With these Skills, candidate action chunks produce predicted future states that the Evaluation Skill scores before dispatching the selected chunk for physical execution. The comparison pathway executes without this imagination-and-selection stage.

insert conservative intermediate verification or repositioning moves to mitigate perceptual uncertainty, which often leads to fragmented and temporally costly trajectories.

User-provided prior knowledge about the policy structure offers a direct remedy to this trade-off. In EMERGE-Policy, such priors are injected into the Main Agent’s active context  $\mathcal{C}_t$  as high-level planning heuristics (e.g., a prescribed action skeleton or explicit constraints on the coupling between `move_to_pose` and `wam_execute` phases). As illustrated in Figure 8, the incorporation of these priors fundamentally alters the generated execution path. In the upper pathway (without prior prompt), the Main Agent, lacking intrinsic knowledge of the task-specific motion coupling, adopts a conservative strategy. It inserts two extra `move_to_pose` commands—one for intermediate visual re-localization and another for redundant pre-position verification—resulting in a zigzag trajectory that prolongs the overall execution time and introduces unnecessary positioning drift.

In contrast, when the prior knowledge is provided (lower pathway of Figure 8), the Main Agent compresses the planning horizon by recognizing that certain movements are implicitly encapsulated within the subsequent `wam_execute` calls. The two superfluous `move_to_pose` cycles are eliminated, enabling a direct and continuous transition between the initial pre-position and the final placement. This streamlined orchestration yields a significantly more fluent execution pattern: the robot motion becomes smoother, with fewer abrupt pauses and less cumulative odometry error. The elimination of redundant waypoints also reduces the cognitive load on the Main Agent, as it no longer needs to evaluate intermediate verification predicates that are already guaranteed by the prior constraints.

Empirically, this prior-guided planning substantially improves the execution metrics in our multi-step cup-stacking tasks. Across repeated trials, the prior-augmented EMERGE-Policy reduces the average number of planning steps per subgoal by 24.7% and decreases the total wall-clock time by 16.3% per episode, while

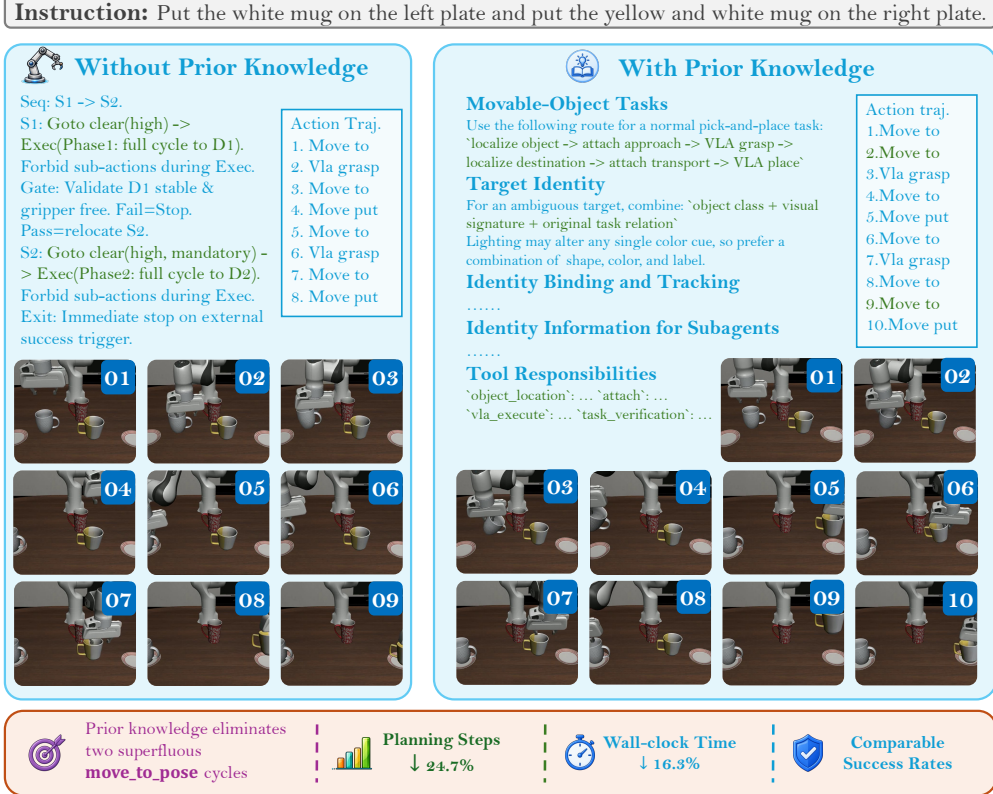


Figure 8: **Effect of task-specific prior knowledge on execution trajectories.** Without the prior, the Main Agent inserts two additional `move_to_pose` commands for re-localization and pre-position verification, producing the upper zigzag trajectory. With the prior, it removes these commands and follows the lower direct trajectory, reducing planning steps by 24.7% and wall-clock time by 16.3% per episode while maintaining comparable success rates.

maintaining a comparable success rate (96.7% versus 95.2% without prior). The reduction in redundant moves is particularly beneficial for long-horizon tasks, where each extra `move_to_pose` introduces additional positioning uncertainty and potential collision risk. These results validate that task-relevant priors enable the Main Agent to better exploit the native capabilities of the Execution Skills, effectively aligning high-level symbolic reasoning with the low-level actuator dynamics. Consequently, the closed-loop system exhibits not only greater robustness—as verified by the Verification Sub Agent—but also a noticeably more anthropomorphic and efficient motion style, as visualized in the contrasting pathways of Figure 8.

## 4 Related Work

**Vision Language Action Models and World Action Models.** Vision Language Action (VLA) models map visual observations and language instructions to robot actions. RT-2<sup>[3]</sup> transfers knowledge from vision language pretraining to robot control, while Open X-Embodiment<sup>[53]</sup>, Octo<sup>[25]</sup>, and OpenVLA<sup>[2]</sup> scale policy learning across tasks and embodiments. Flow-based policies such as  $\pi_0$ <sup>[6]</sup> and  $\pi_{0.5}$ <sup>[7]</sup> further improve expressive action generation. These models provide strong visuomotor execution, but a policy rollout alone does not explicitly organize task decomposition, outcome verification, recovery, and persistent task state. EMERGE-Policy is complementary to this line of work: it uses a VLA as an Operational Skill within a broader orchestration loop rather than replacing or retraining the underlying policy. World models provide a related capability by predicting future observations or latent states under candidate actions. RoboDreamer<sup>[54]</sup> studies generative robot imagination, while VLMPC<sup>[55]</sup> and World Action Verifier<sup>[56]</sup> connect prediction to

action selection or rollout assessment. Recent World Action Models can also generate actions, making the boundary between prediction and control increasingly dependent on how a model is invoked. EMERGE-Policy therefore categorizes capabilities by their function in the decision loop: an invocation that produces executable actions is an Operational Skill, one that predicts candidate outcomes is an Imagination Skill, and one that assesses feasibility, progress, or completion is an Evaluation Skill. This functional interface allows VLA and world-model backends to participate in the same closed-loop protocol without conflating their system roles.

**Agent and Harness Engineering.** Language-model agents for robotics commonly connect high-level reasoning to callable perception and control interfaces. Code as Policies<sup>[16]</sup> and ProgPrompt<sup>[57]</sup> express robot behavior as executable programs, and RoboCodeX<sup>[17]</sup> extends program generation with multimodal inputs. CaP-X<sup>[18]</sup> further shows that primitive abstraction, perceptual grounding, and interaction design materially affect coding-agent reliability. These studies establish that robot performance depends not only on the reasoning model, but also on the interface that constrains actions, refreshes state, and returns execution evidence. Agent engineering has also introduced mechanisms for learning from feedback and maintaining state<sup>[58]</sup>. Reflexion<sup>[23]</sup> converts task feedback into textual reflection, Statler<sup>[59]</sup> maintains an explicit world state across extended decision sequences, and ASPIRE<sup>[19]</sup> diagnoses failures, validates repairs, and consolidates successful procedures into reusable robot skills. EMERGE-Policy draws on these ideas but addresses a different systems question: how should heterogeneous evidence be distributed during online robot execution? Task planning primarily requires instruction, plan status, and concise task state, whereas perception, monitoring, verification, and memory consolidation may involve dense observations and intermediate processing traces. EMERGE-Policy assigns these responsibilities to role-specific Sub Agents with isolated contexts and structured outputs, so that the Main Agent receives the evidence needed for the current decision without retaining every processing trace in its active context window. The contribution is therefore an explicit organization of agent roles, information boundaries, and handovers, rather than a new language-model learning rule.

**Hierarchical and Closed-Loop Robot Agents.** Language-guided robot systems have long separated high-level decisions from low-level skills. SayCan<sup>[60]</sup> combines language-model scores with learned affordances, Inner Monologue<sup>[61]</sup> incorporates environment feedback into replanning, and VoxPoser<sup>[62]</sup> grounds language constraints as three-dimensional value maps. Hi Robot<sup>[63]</sup> and Agentic Robot<sup>[64]</sup> further combine deliberative reasoning with learned execution. These works demonstrate the value of hierarchy and feedback; consequently, neither hierarchy nor closed-loop control alone is the distinguishing claim of EMERGE-Policy.

Recent robot harnesses provide the closest comparison. RoboClaw<sup>[20]</sup> combines a vision language meta-controller, structured memory, and a stack of Skills, Tools, and Policies. HarnessVLA<sup>[21]</sup> uses a memory-guided planner to coordinate a frozen contact-rich VLA primitive with analytical controllers, while RoboHarness<sup>[22]</sup> emphasizes runtime orchestration around robot policies. RoboOS<sup>[65]</sup> combines global planning, embodiment-specific skills, and shared memory for cross-embodiment and multi-agent operation. These systems differ in scope and implementation, but primarily organize which capability executes a subtask and how control passes across planning and execution. EMERGE-Policy focuses additionally on the information plane: perception, monitoring, verification, and memory processing are assigned to explicit Sub-Agent roles, each with its own task-scoped context and structured handover to the Main Agent. This design complements capability-level orchestration by making the provenance and destination of decision-relevant evidence explicit, while keeping role-specific traces outside the active planner context until they are needed.

A complementary line of work studies supervision, outcome assessment, and recovery. LITEN<sup>[66]</sup> supports feedback-driven replanning, VLAC<sup>[67]</sup> estimates progress and completion from visual trajectories, and PhyAgentOS<sup>[68]</sup> separates an Agent plane from a supervised Runtime whose verifier evaluates evidence against acceptance criteria. RoboClaw monitors subtask state and selects retry or recovery strategies, while HarnessVLA verifies primitive outcomes before restaging or retrying. EMERGE-Policy integrates related functions through Evaluation Skills and role-specific Verification and Monitor Sub Agents. When an observed outcome violates the active subgoal criterion, the returned evidence supports a textual diagnosis and a localized recovery subgoal on the Branch Stack; the Main Agent revises the global plan only when local recovery is insufficient. Thus, the framework brings capability invocation, evidence processing, verification, recovery, and external memory into one agentic orchestration graph; the resulting system-level policy is shaped by the interaction of these roles rather than by any single backend.

## 5 Conclusion and Future Work

We present EMERGE-Policy, a graph-structured agentic policy framework in which robot behavior emerges from the coordinated operation of perception, prediction, execution, verification, recovery, and memory. In this sense, EMERGE-Policy is not another low-level control policy; it provides the structure through which interactions among heterogeneous components become coherent system-level behavior. The Main Agent maintains compact task-level state and orchestrates a role-based skill interface, while isolated Sub Agents process dense perceptual and execution evidence. Operational, Imagination, and Evaluation Skills provide a common abstraction for action-producing policies, future-predictive models, and outcome assessment, and verified failures are routed through localized Branch Stack recovery before global replanning.

Across the reported LIBERO evaluations, the complete system improves over its underlying  $\pi_{0.5}$  and Cosmos Policy execution backends, with particularly large gains under several perturbation and implicit-instruction settings. These are system-level results: establishing which gains arise from role-specific context decomposition, imagination, verification, memory, or increased observation and interaction budgets requires the matched ablations identified in [Section 3](#). The current real-robot cup-stacking study further illustrates multistep physical deployment, but its statistical strength depends on reporting repeated trials, intervention rules, execution cost, and failure categories. Future work should make the harness evaluation protocol reproducible, quantify trade-offs among accuracy, latency, and token use, and study how the same fixed orchestration graph transfers across planners, low-level policies, tasks, and embodiments. A particularly important direction is to distinguish safe within-episode adaptation from persistent cross-episode learning, enabling experience reuse without contaminating benchmark evaluation.

Future work should make the harness evaluation protocol reproducible, quantify accuracy–latency–token trade-offs, and study how the same fixed orchestration graph transfers across planners, low-level policies, tasks, and embodiments. A particularly important direction is to distinguish safe within-episode adaptation from persistent cross-episode learning, enabling experience reuse without contaminating benchmark evaluation.

## References

- [1] Annie Murphy Paul. The extended mind: The power of thinking outside the brain. Mariner Books, 2021.
- [2] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. arXiv preprint arXiv:2406.09246, 2024.
- [3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. arXiv preprint arXiv:2307.15818, 2023.
- [4] Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, et al. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. arXiv preprint arXiv:2411.19650, 2024.
- [5] Moritz Reuss, Ömer Erdiñç Yağmurlu, Fabian Wenzel, and Rudolf Lioutikov. Multimodal diffusion transformer: Learning versatile behavior from multimodal goals. arXiv preprint arXiv:2407.05996, 2024.
- [6] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al.  $\pi_0$ : A vision-language-action flow model for general robot control. arXiv preprint arXiv:2410.24164, 2024.
- [7] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al.  $\pi_{0.5}$ : a vision-language-action model with open-world generalization. arXiv preprint arXiv:2504.16054, 2025.
- [8] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. arXiv preprint arXiv:2501.09747, 2025.
- [9] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. arXiv preprint arXiv:2503.14734, 2025.
- [10] Chi-Lam Cheang, Guangzeng Chen, Ya Jing, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Hongtao Wu, Jiafeng Xu, Yichu Yang, et al. Gr-2: A generative video-language-action model with web-scale knowledge for robot manipulation. arXiv preprint arXiv:2410.06158, 2024.
- [11] Yichao Shen, Fangyun Wei, Zhiying Du, Yaobo Liang, Yan Lu, Jiaolong Yang, Nanning Zheng, and Baining Guo. Videovla: Video generators can be generalizable robot manipulators. Advances in neural information processing systems, 38:95597–95621, 2026.
- [12] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. arXiv preprint arXiv:2412.14803, 2024.
- [13] Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, et al. Cosmos policy: Fine-tuning video models for visuomotor control and planning. arXiv preprint arXiv:2601.16163, 2026.
- [14] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, et al. World action models are zero-shot policies. arXiv preprint arXiv:2602.15922, 2026.
- [15] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-wam: Do world action models need test-time future imagination? arXiv preprint arXiv:2603.16666, 2026.
- [16] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In 2023 IEEE International conference on robotics and automation (ICRA), pages 9493–9500. IEEE, 2023.
- [17] Yao Mu, Junting Chen, Qinglong Zhang, Shoufa Chen, Qiaojun Yu, Chongjian Ge, Runjian Chen, Zhixuan Liang, Mengkang Hu, Chaofan Tao, et al. Robocodex: Multimodal code generation for robotic behavior synthesis. arXiv preprint arXiv:2402.16117, 2024.

- [18] Letian Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Guanzhi Wang, Dantong Niu, Fei-Fei Li, et al. Cap-x: A framework for benchmarking and improving coding agents for robot manipulation. [arXiv preprint arXiv:2603.22435](#), 2026.
- [19] Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, Ken Goldberg, Ang Chen, et al. Aspire: Agentic/skills discovery for robotics. [arXiv preprint arXiv:2607.00272](#), 2026.
- [20] Ruiying Li, Yunlang Zhou, YuYao Zhu, Kylin Chen, Jingyuan Wang, Sukai Wang, Kongtao Hu, Minhui Yu, Bowen Jiang, Zhan Su, et al. Roboclaw: An agentic framework for scalable long-horizon robotic tasks. [arXiv preprint arXiv:2603.11558](#), 2026.
- [21] Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Chunyang Zhu, Jiaying Qiu, Yuchen Yan, Jiyuan Liu, Wenhao Tang, et al. Harness vla: Steering frozen vlas into reliable manipulation primitives via memory-guided agents. [arXiv preprint arXiv:2607.08448](#), 2026.
- [22] Jinbang Huang, Yuanzhao Hu, Zhiyuan Li, Ran Qi, Yixin Xiao, Zhanguang Zhang, Mark Coates, Tongtong Cao, and Yingxue Zhang. Roboharness: Memory-driven orchestration of heterogeneous robot policies for long-horizon planning. [arXiv preprint arXiv:2607.18060](#), 2026.
- [23] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. [Advances in neural information processing systems](#), 36:8634–8652, 2023.
- [24] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. [The International Journal of Robotics Research](#), 44(10-11):1684–1704, 2025.
- [25] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. [arXiv preprint arXiv:2405.12213](#), 2024.
- [26] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. [arXiv preprint arXiv:2502.19645](#), 2025.
- [27] Jinhui Ye, Ning Gao, Senqiao Yang, Jinliang Zheng, Zixuan Wang, Yuxin Chen, Pengguang Chen, Yilun Chen, Shu Liu, and Jiaya Jia. Starvla- $\alpha$ : Reducing complexity in vision-language-action systems. [arXiv preprint arXiv:2604.11757](#), 2026.
- [28] Likui Zhang, Tao Tang, Zhihao Zhan, Xiuwei Chen, Zisheng Chen, Jianhua Han, Jiangtong Zhu, Pei Xu, Hang Xu, Hefeng Wu, et al. Atomicvla: Unlocking the potential of atomic skill learning in robots. [arXiv preprint arXiv:2603.07648](#), 2026.
- [29] Junyi Zhang, Jiabin Ge, Hanjun Yoo, Letian Fu, Zihan Yang, Yaowei Liu, Raj Saravanan, Shaofeng Yin, Justin Yu, Dantong Niu, et al. Playful agentic robot learning. [arXiv preprint arXiv:2606.19419](#), 2026.
- [30] Qingwen Bu, Yanting Yang, Jisong Cai, Shenyuan Gao, Guanghui Ren, Maoqing Yao, Ping Luo, and Hongyang Li. Univla: Learning to act anywhere with task-centric latent actions. [arXiv preprint arXiv:2505.06111](#), 2025.
- [31] Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, Xin Li, Yibing Song, Hao Luo, Fan Wang, et al. Worldvla: Towards autoregressive action world model. [arXiv preprint arXiv:2506.21539](#), 2025.
- [32] Shuhan Tan, Kairan Dou, Yue Zhao, and Philipp Krähenbühl. Interactive post-training for vision-language-action models. [arXiv preprint arXiv:2505.17016](#), 2025.
- [33] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, Xinqiang Yu, Jiazhaohao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, et al. Dreamvla: a vision-language-action model dreamed with comprehensive world knowledge. [Advances in Neural Information Processing Systems](#), 38:24195–24228, 2026.
- [34] Yandan Yang, Shuang Zeng, Tong Lin, Xinyuan Chang, Dekang Qi, Junjin Xiao, Haoyun Liu, Ronghan Chen, Yuzhi Chen, Dongjie Huo, et al. Abot-m0: Vla foundation model for robotic manipulation with action manifold learning. [arXiv preprint arXiv:2602.11236](#), 2026.
- [35] Hao Luo, Wanpeng Zhang, Yicheng Feng, Sipeng Zheng, Haiweng Xu, Chaoyi Xu, Ziheng Xi, Yuhui Fu, and Zongqing Lu. Being-h0. 7: A latent world-action model from egocentric videos. [arXiv preprint arXiv:2605.00078](#), 2026.



- [36] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, et al. X-vla: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. In *International Conference on Learning Representations*, volume 2026, pages 60580–60606, 2026.
- [37] Lin Li, Qihang Zhang, Yiming Luo, Shuai Yang, Ruilin Wang, Fei Han, Mingrui Yu, Zelin Gao, Nan Xue, Xing Zhu, et al. Causal world modeling for robot control. *arXiv preprint arXiv:2601.21998*, 2026.
- [38] Jason Lee, Jiafei Duan, Haoquan Fang, Yuquan Deng, Shuo Liu, Boyang Li, Bohan Fang, Jieyu Zhang, Yi Ru Wang, Sangho Lee, et al. Molmoact: Action reasoning models that can reason in space. *arXiv preprint arXiv:2508.07917*, 2025.
- [39] Yicheng Liu, Zibin Dong, Baijun Ye, Tianyuan Yuan, Tao Jiang, et al. G0.5: One autoregressive stream for robot reasoning and action. *arXiv preprint arXiv:2608.11739*, 2026.
- [40] Xiaomi Robotics Team, Jun Guo, Piaopiao Jin, Jason Li, Peiyan Li, Yingyan Li, Futeng Liu, Wanli Peng, Optimus Qin, Yifei Su, et al. Xiaomi-robotics-1: Scaling vision-language-action models with over 100k hours of real-world trajectories. *arXiv preprint arXiv:2607.15330*, 2026.
- [41] He Zhang, Lingzhu Xiang, Haitao Lin, Zeyu Huang, Minghui Wang, Dingyan Zhong, Yubo Dong, Yihao Wu, Yongming Rao, Dongsheng Zhang, et al. Hy-embodied-0.5-vla: From vision-language-action models to a real-world robot learning stack. *arXiv preprint arXiv:2606.14409*, 2026.
- [42] Fuhao Li, Wenxuan Song, Han Zhao, Jingbo Wang, Pengxiang Ding, Donglin Wang, Long Zeng, and Haoang Li. Spatial forcing: Implicit spatial representation alignment for vision-language-action model. In *International Conference on Learning Representations*, volume 2026, pages 132324–132345, 2026.
- [43] Haoxiang Ma, Junhao Cai, Xiaoxu Xu, Hao Li, Yuyin Yang, Yang Tian, Jiafei Cao, Hongrui Zhu, Zherui Qiu, Yuqiang Yang, et al. Internvla-a1. 5: Unifying understanding, latent foresight, and action for compositional generalization. *arXiv preprint arXiv:2607.04988*, 2026.
- [44] Senqiao Yang, Jinhui Ye, Pengguang Chen, and Shu Liu. Vllact qwenoft qwen3-vl-4b for robodojo. Hugging Face model card, 2026.
- [45] Jun Guo, Qiwei Li, Peiyan Li, Zilong Chen, Nan Sun, Yifei Su, Heyun Wang, Yuan Zhang, Xinghang Li, and Huaping Liu. Unified 4d world action modeling from video priors with asynchronous denoising. *arXiv preprint arXiv:2604.26694*, 2026.
- [46] Rui Cai, Jun Guo, Xinze He, Piaopiao Jin, Jie Li, Bingxuan Lin, Futeng Liu, Wei Liu, Fei Ma, Kun Ma, et al. Xiaomi-robotics-0: An open-sourced vision-language-action model with real-time execution. *arXiv preprint arXiv:2602.12684*, 2026.
- [47] Angen Ye, Boyuan Wang, Chaojun Ni, Guan Huang, Guosheng Zhao, Hao Li, Hengtao Li, Jie Li, Jindi Lv, Jingyu Liu, et al. Gigaworld-policy: An efficient action-centered world-action model. *arXiv preprint arXiv:2603.17240*, 2026.
- [48] Tao Jiang, Tianyuan Yuan, Yicheng Liu, Chenhao Lu, Jianning Cui, Xiao Liu, Shuiqi Cheng, Jiyang Gao, Huazhe Xu, and Hang Zhao. Galaxea open-world dataset and g0 dual-system vla model. *arXiv preprint arXiv:2509.00576*, 2025.
- [49] Wei Wu, Fan Lu, Yunnan Wang, Shuai Yang, Shi Liu, Fangjing Wang, Qian Zhu, He Sun, Yong Wang, Shuailei Ma, et al. A pragmatic vla foundation model. *arXiv preprint arXiv:2601.18692*, 2026.
- [50] Ganlin Yang, Zhangzheng Tu, Yuqiang Yang, Sitong Mao, Junyi Dong, Tianxing Chen, Jiaqi Peng, Jing Xiong, Jiafei Cao, Jifeng Dai, et al. Eventvla: Event-driven visual evidence memory for long-horizon vision-language-action policies. *arXiv preprint arXiv:2606.20092*, 2026.
- [51] Jisong Cai, Long Ling, Shiwei Chu, Zhongshan Liu, Jiayue Kang, Zhixuan Liang, Wenjie Xu, Yinan Mao, Weinan Zhang, Xiaokang Yang, et al. Aha-wam: Asynchronous horizon-adaptive world-action modeling with observation-guided context routing. *arXiv preprint arXiv:2606.09811*, 2026.
- [52] Tianxing Chen, Yue Chen, Zixuan Li, et al. Robodojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies. *arXiv preprint arXiv:2607.04434*, 2026.
- [53] Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets

- and rt-x models: Open x-embodiment collaboration 0. In 2024 IEEE International Conference on Robotics and Automation (ICRA), pages 6892–6903. IEEE, 2024.
- [54] Siyuan Zhou, Yilun Du, Jiaben Chen, Yandong Li, Dit-Yan Yeung, and Chuang Gan. Robodreamer: Learning compositional world models for robot imagination. arXiv preprint arXiv:2404.12377, 2024.
  - [55] Wentao Zhao, Jiaming Chen, Ziyu Meng, Donghui Mao, Ran Song, and Wei Zhang. Vlmpe: Vision-language model predictive control for robotic manipulation. arXiv preprint arXiv:2407.09829, 2024.
  - [56] Yuejiang Liu, Fan Feng, Lingjing Kong, Weifeng Lu, Jinzhou Tang, Kun Zhang, Kevin Murphy, Chelsea Finn, and Yilun Du. World action verifier: Self-improving world models via forward-inverse asymmetry. arXiv preprint arXiv:2604.01985, 2026.
  - [57] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. arXiv preprint arXiv:2209.11302, 2022.
  - [58] Weihao Tan, Wentao Zhang, Xinrun Xu, Haochong Xia, Ziluo Ding, Boyu Li, Bohan Zhou, Junpeng Yue, Jiechuan Jiang, Yewen Li, et al. Cradle: Empowering foundation agents towards general computer control. arXiv preprint arXiv:2403.03186, 2024.
  - [59] Takuma Yoneda, Jiading Fang, Peng Li, Huanyu Zhang, Tianchong Jiang, Shengjie Lin, Ben Picker, David Yunis, Hongyuan Mei, and Matthew R Walter. Statler: State-maintaining language models for embodied reasoning. In 2024 IEEE International Conference on Robotics and Automation (ICRA), pages 15083–15091. IEEE, 2024.
  - [60] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. arXiv preprint arXiv:2204.01691, 2022.
  - [61] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. arXiv preprint arXiv:2207.05608, 2022.
  - [62] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. arXiv preprint arXiv:2307.05973, 2023.
  - [63] Lucy Xiaoyang Shi, Brian Ichter, Michael Equi, Liyiming Ke, Karl Pertsch, Quan Vuong, James Tanner, Anna Walling, Haohuan Wang, Niccolo Fusai, et al. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. arXiv preprint arXiv:2502.19417, 2025.
  - [64] Zhejian Yang, Yongchao Chen, Xueyang Zhou, Jiangyue Yan, Dingjie Song, Yinuo Liu, Yuting Li, Yu Zhang, Pan Zhou, Hechang Chen, et al. Agentic robot: A brain-inspired framework for vision-language-action models in embodied agents. arXiv preprint arXiv:2505.23450, 2025.
  - [65] Huajie Tan, Xiaoshuai Hao, Cheng Chi, Minglan Lin, Yaoxu Lyu, Mingyu Cao, Dong Liang, Zhuo Chen, Mengsi Lyu, Cheng Peng, et al. Roboos: A hierarchical embodied framework for cross-embodiment and multi-agent collaboration. arXiv preprint arXiv:2505.03673, 2025.
  - [66] Ameesh Shah, William Chen, Adwait Godbole, Federico Mora, Sanjit A Seshia, and Sergey Levine. Learning affordances at inference-time for vision-language-action models. arXiv preprint arXiv:2510.19752, 2025.
  - [67] Shaopeng Zhai, Qi Zhang, Tianyi Zhang, Fuxian Huang, Haoran Zhang, Ming Zhou, Shengzhe Zhang, Litao Liu, Sixu Lin, and Jiangmiao Pang. A vision-language-action-critic model for robotic real-world reinforcement learning. arXiv preprint arXiv:2509.15937, 2025.
  - [68] Yang Liu, Weixing Chen, Xinshuai Song, Tao Pu, Siwen Mo, Yongjie Bai, Zihao Chen, Qianran Sun, Liruo Zhong, Ying Shen, et al. Phyagentos: A self-evolving operating system for embodied agents with decoupled cognitive planning and physical execution. arXiv preprint arXiv:2607.16636, 2026.